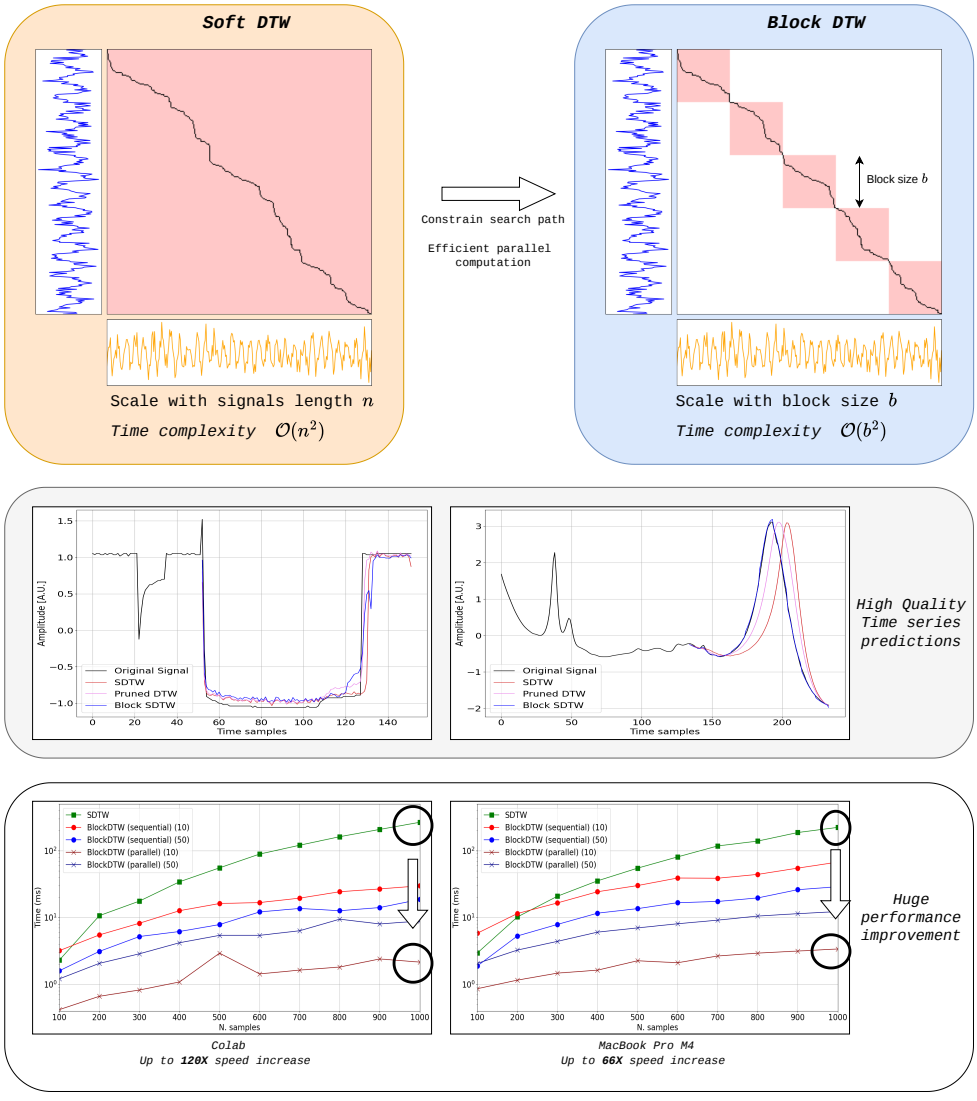
Graphical Abstract

BlockDTW: Efficient, Parallel and Differentiable Time-Series Alignment for Neural Learning Systems

Alberto Zancanaro, Leonardo Badia, Giulia Cisotto



Highlights

BlockDTW: Efficient, Parallel and Differentiable Time-Series Alignment for Neural Learning Systems

Alberto Zancanaro, Leonardo Badia, Giulia Cisotto

- BlockDTW is a differentiable temporal-alignment loss for neural learning.
- Block-wise alignment reduces quadratic complexity to linear sequence scaling.
- BlockDTW supports parallel gradient-based training on long time-series.
- Neural predictors achieve comparable quality across 73 UCR datasets.
- Parallel BlockDTW reaches up to 120× speedup over SoftDTW.

BlockDTW: Efficient, Parallel and Differentiable Time-Series Alignment for Neural Learning Systems

Alberto Zancanaro[a], Leonardo Badia[b], Giulia Cisotto[c]

[a]*Luxembourg Centre for Systems Biomedicine, 6 Av. du Swing, Esch-Belval Sanem, 4367, Luxembourg*
[b]*Department of Information Engineering, University of Padova, via Gradenigo 6/b, Padova, 35121, Italy*
[c]*Dept. Mathematics, Informatics, and Geosciences, University of Trieste, via Economo 12/3, Trieste, 34123, Italy*

Abstract

Dynamic Time Warping (DTW) is a cornerstone technique for measuring similarity between time-series under temporal distortions, but its quadratic time and space complexity severely limits scalability, real-time deployment, and its use as a loss function in neural time-series learning systems. Several approximations have been proposed to address these limitations, often trading off accuracy, differentiability, or computational efficiency. In this work, we introduce BlockDTW, a block-wise, differentiable temporal-alignment loss that partitions time-series into non-overlapping segments and computes local alignments independently. The local alignment terms are aggregated into a global learning objective, preserving compatibility with gradient-based optimization while enabling efficient neural-network training. By constraining the admissible warping paths to block junctions (or *islands*), BlockDTW reduces computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(bN)$, while its parallel implementation processes the independent blocks concurrently. We evaluate BlockDTW across three representative tasks: (i) similarity measurement and alignment of synthetic frequency-varying sinusoids, (ii) time-series prediction using feedforward neural networks on the Trace dataset and 73 datasets from the UCR Archive, and (iii) training a deep neural model (hvEEGNet) for high-fidelity reconstruction of multichannel EEG signals. Experimental results show that BlockDTW provides a good approximation of Soft-DTW, achieves prediction and reconstruction performance comparable to established differentiable alignment losses, and significantly reduces computational cost, with hardware-dependent speedups reaching up to 120× in

dedicated parallel-runtime benchmarks. These findings establish BlockDTW as an efficient and scalable differentiable learning objective for neural models operating on temporally distorted sequences, with particular potential for real-time and biomedical learning applications.

Keywords: Differentiable loss, Neural networks, Time-series learning, Dynamic time warping, Temporal alignment, Parallel computing, EEG

---

1. Introduction

Dynamic Time Warping (DTW) has long been regarded as the gold standard for aligning time-series affected by temporal distortions, as it explicitly computes an optimal warping path between signals [1, 2, 3]. Despite its robustness, the original formulation of DTW suffers from quadratic time and space complexity, making it impractical for long sequences, large datasets, and real-time or learning-based scenarios. This limitation is particularly relevant in neural learning systems, where differentiable alignment measures may be evaluated repeatedly during backpropagation over large training sets. Their computational cost directly affects training time and feasible sequence length and, when alignment is also required online, the real-time feasibility of the resulting system. These requirements arise across several application domains, including healthcare and wellbeing, for example in stress-management systems [4]; Industry 5.0 and Digital Twins [5]; and autonomous vehicles, for trajectory and sensor-signal alignment [6]. In all these settings, temporal similarity measures must be computed efficiently during model training and/or online inference.

To mitigate these issues, numerous DTW variants and approximations have been proposed. FastDTW [7] and pruning-based approaches reduce computation by constraining the search space, but may miss optimal alignments under complex distortions. SoftDTW [8] introduces differentiability by replacing the hard minimum with a smooth approximation, enabling its use as a loss function in neural networks, but it retains quadratic complexity. More recent approaches such as Optimal Transport Warping (OTW) [9] aim at linear complexity and differentiability, yet often sacrifice interpretability or fine-grained temporal alignment.

In many real-world contexts, particularly biomedical ones such as EEG, ECG, or EMG, relevant events are expected to occur within limited temporal neighborhoods, while large-scale misalignments are either unlikely or

undesirable (e.g., artefacts aligning with physiological events) [10, 11]. This observation motivates a block-wise abstraction of the alignment problem, where strong assumptions on global warping are relaxed in favor of locally constrained, computationally efficient matching.

Building on this insight, we propose Block-wise DTW (BlockDTW), an approximation of BlockDTW that divides time-series into non-overlapping segments and computes local SoftDTW scores independently, summing them to obtain a global similarity measure. This formulation significantly reduces computational cost, enables parallel execution, and preserves differentiability, making it suitable as a differentiable training objective for neural models that predict, reconstruct, or otherwise learn temporally structured sequences. In the conference paper [12], we first introduced BlockDTW and presented preliminary results demonstrating its promising potential. The present article substantially extends that work by providing a more comprehensive description of the method, an expanded discussion of the state of the art, large-scale benchmarking on the UCR Archive, a parallel implementation of the algorithm, and an additional discussion of its limitations and applicability. In addition, we publicly released the code for BlockDTW as a Python package on GitHub[1] and PyPI[2].

From a neural-learning perspective, the main contributions of this work are threefold. First, we formulate BlockDTW as a differentiable temporal-alignment objective that can replace quadratic-complexity SoftDTW losses in gradient-based learning. Second, we evaluate its effect on neural prediction and reconstruction across heterogeneous time-series, including 73 UCR datasets and multichannel EEG recordings. Third, we provide sequential and parallel implementations and characterize the resulting accuracy-efficiency trade-off across different hardware platforms.

The remainder of the paper is organized as follows. Section 2 reviews the foundations of Dynamic Time Warping and the most relevant related approaches. Section 3 introduces the proposed BlockDTW method and describes the experimental setup, three case studies, and implementation details. Section 4 presents the experimental results and discusses the performance across the considered case studies. Finally, Section 5 concludes the paper and outlines directions for future research.

[1] `https://github.com/jesus-333/dtw_loss_functions`

[2] `https://pypi.org/project/dtw-loss-functions/`

2. Dynamic Time Warping

2.1. Foundational Work

Let us define $x[i]$ and $y[j]$ as two time-series with N and M samples, respectively, $i = 1, 2, \ldots, N$, and $j = 1, 2, \ldots, M$. The problem is to compute a measure of their similarity (or, their distance). A recent survey of the most commonly used distance measures is available in [13]. Euclidean distance, for example, is fundamental to compute the mean square error (MSE) and its variants (root MSE, mean absolute error) between x and y: these methods sum the (squared) difference between corresponding time points, giving an aggregated measure of their time-to-time similarity. Note that this assumes that the two time series have the same length (or, the last samples of the longer sequence are discarded). *Cross-correlation function*, on the other hand, quantifies the similarity between two time-series as the integral of their point-wise product, at any given shift in time of the second one w.r.t. the first. *Cosine similarity* has also been employed to quantify adherence of one time-series to another. However, it suffers from limitations for time-series and is confined to specific domains (e.g., information retrieval [14]). All the above measures turned out to be very sensitive to small distortions or shifts of one time-series w.r.t. the other.

Thus, in the 1970s the dynamic programming algorithm called DTW was introduced [1]. The idea of dynamically aligning two time-series was first conceptualized in 1975 in [2], then brought to its modern version by [1], and finally popularized by [3] for time-series analysis.

The algorithm operates in three main steps: (i) a cost matrix Δ is created, associating its rows and columns to the amplitude values of $x(i)$ and $y(j)$, respectively. (ii) Starting from $\Delta(0, 0)$, the value of each element is computed as

$$\begin{aligned} \Delta(i, j) = \delta(x[i], y[j]) \; + \min[\Delta(i-1, j-1), \\ + \Delta(i, j-1), \Delta(i-1, j)], \end{aligned} \tag{1}$$

if $i, j \geq 1$, otherwise $\Delta(i, j) = \delta(x(i), y(j))$, where $\delta(\cdot)$ is a distance function.

(iii) The final DTW score is the lower right corner $\Delta(N, M)$ of the cost matrix. A pseudocode for this procedure is reported in Algorithm 1.

Formally, DTW is computed as follows:

$$\mathrm{DTW}(x, y) = \min_{A \in \mathcal{A}_{n,m}} \left(\langle A, \Delta(x, y) \rangle \right) \tag{2}$$

Algorithm 1 Full DTW Algorithm

$x \in \mathbb{R}^N, y \in \mathbb{R}^M$
$\Delta_{0,0} = 0, \Delta_{0,1} = 0, \Delta_{1,0} = 0$
$\Delta_{i,0} = \infty, \Delta_{0,j} = \infty$ for $i, j \geq 2$
for $(i = 1; i \leq N; i = i + 1)$ **do**
  for $(j = 1; j \leq M; j = j + 1)$ **do**
    $\Delta_{\min} = \min(\Delta_{i-1,j}, \Delta_{i-1,j-1}, \Delta_{i,j-1})$
    $\Delta_{i,j} = \delta(x_i, y_j) + \Delta_{\min}$
  end for
end for
Return $\Delta_{N,M}$

with $\Delta(x, y)$ as the distance matrix defined in Algorithm 1, $\mathcal{A}_{n,m} \subset \{0, 1\}^{N \times M}$ as the set of binary alignment matrices [8], i.e., the matrices that contain a path from the upper left corner to the lower right corner, and $\langle \cdot, \cdot \rangle$ as the inner product. To create a path, $\rightarrow, \downarrow, \searrow$ steps are allowed, only. The overall DTW *score* between time-series x and y is given by the output of Algorithm 1.

This algorithm has been applied in many different domains and has provided significant advantages compared to other measures of similarity, especially for its robustness to warping, distortion, and shift, enabling data analytics solutions to work with real-world time-series [15]. Unfortunately, in its original formulation (called hereafter *Full DTW*), DTW has both time and space complexity of $\mathcal{O}(NM)$. Since $N \approx M$, as a matter of fact, DTW scales as $\mathcal{O}(N^2)$ in time and memory. it is worth noting that if the warping path is of no interest, then complexity reduces to $\mathcal{O}(N)$.

Since its first publication, many different versions have been proposed to address computational efficiency, without excessively sacrificing other important requirements such as accuracy, scalability, differentiability, non-negativity, and real-time constraints for applications like Brain-Computer Interface (BCI) [16].

2.2. State of the art in DTW computation

A number of algorithms have been proposed so far to improve the computational complexity of the original DTW. Here, we summarize those that most closely align with our purposes.

FastDTW [17] is a linear-time and space approximation of DTW, designed to scale by recursively computing DTW on downsampled series and refining the path within a constrained window. It achieves very good DTW

approximation with runtime reductions of several orders of magnitude (complexity O(N)), but may miss optimal paths in highly non-linear alignments. Despite widespread use, recent works [7] question its real efficiency.

PruneDTW [18] accelerates the exact computation of DTW by exploiting Keogh's lower bound [19] to prune unlikely cells from the cost matrix, i.e., it uses this bound to reduce the number of admissible alignment paths and thereby decreases the overall computational cost. Moreover, the method can be combined with a global constraint in the form of a restriction band, an a priori narrowing of the feasible path search region. Well-known examples of such bands are the Sakoe–Chiba band [1] and the Itakura parallelogram [2]. On UCR datasets, it achieved very good DTW approximation with up to 20× speedups, though its performance drops when sequences are highly distorted.

Stochastic DTW [20] offers a probabilistic ensemble of alignment paths, enabling uncertainty modeling (e.g., for seismic data). While robust to noise, its reliance on sampling increases computational cost, limiting real-time and deep learning integration.

OTW [9] is a time-series similarity measure that frames alignment as a problem in the Optimal Transport (OT) domain. It treats each time series as a distribution of "mass" and computes a transport cost to map one to the other. OTW is designed to be linear in time/space complexity, differentiable, and parallelizable on modern hardware, making it suitable for integration into machine learning pipelines where traditional DTW becomes a computational bottleneck. OTW also exhibits moderate sensitivity to temporal and shape distortions, which helps it capture meaningful differences between time series without incurring the high cost of exact alignment matrices. Despite these advantages, OTW has several limitations: the method's emphasis on moderate sensitivity can limit its flexibility in scenarios where finer or stronger temporal elastic alignment is desired, potentially reducing alignment fidelity for highly non-linear time distortions; OTW produces a coupling (transport plan) rather than an explicit, interpretable alignment path, which may complicate analyses that require discrete warp paths as outputs (as for clinical applications); implementation involves algorithmic choices, regularization, and numeric settings that affect performance and stability, requiring careful tuning in real applications and potentially introducing trade-offs between accuracy and computational cost.

SoftDTW [8] is the most common differentiable DTW approximation, reformulating DTW as a smooth loss by replacing the minimum in (2) with a soft-min operator.

Given a series of n numbers, $[a_1, ..., a_n]$, the *soft* minimum operation is defined as

$$\min^{\gamma}(a_1, ..., a_n) = -\gamma \log \sum_{i=1}^{n} e^{-a_i/\gamma} \tag{3}$$

with $\gamma > 0$ being a smoothing hyperparameter. The lower γ is, the closer (3) is to the minimum. Then, SoftDTW is defined as

$$\begin{aligned} \mathrm{SDTW}_{\gamma}(x, y) &= \min^{\gamma}(\langle A, \Delta(x, y)\rangle, A \in \mathcal{A}_{n,m}) && (4) \\ &= -\gamma \log \sum_{A \in \mathcal{A}_{n,m}} e^{\langle A, \Delta(x,y)\rangle/\gamma} && (5) \end{aligned}$$

With SoftDTW, each sample of a time-series is compared with every sample of the other. This means that the memory usage and the number of computations scale as $\mathcal{O}(N^2)$.

DTW divergence (DTW-div) is simply derived from any measure of DTW by the following formula:

$$D_{\gamma}(x, y) = \mathrm{DTW}_{\gamma}(x, y) - \frac{1}{2}(\mathrm{DTW}_{\gamma}(x, x) + \mathrm{DTW}_{\gamma}(y, y)) \tag{6}$$

DTW-div ensures non-negativity and minimization when the two time-series are very similar, at the cost of a higher computation time than a single DTW computation. Specifically, DTW-div is affected by a factor 3, as DTW needs to be computed three times (this could be overcome by parallel computation).

In [9], the authors proposed the OTW algorithm which showed linear time and space complexity and differentiability. Also, the authors claimed that the algorithm was parallelized. However, performance was tested on datasets different from those used in this work (a simpler synthetic dataset including triangle or square signals with some amount of time shift, plus white noise) and different tasks (specifically, classification via deep learning and 1-nearest neighbor clustering, and hierarchical clustering), making it difficult to fairly compare the two works.

A very recent article introduced *TimePoint* [24], an algorithm whose alignment strategy relies on discrete rules (keypoint extraction and matching). While promising, this method does not allow differentiation or backpropagation. Table 1 reports a list of the most relevant and recent algorithms for DTW computation. They are compared based on their time and space complexity, approximation quality w.r.t. the Full DTW (assuming DTW gives the best distance path between the time-series, i.e., distance

Table 1: State of the art and BlockDTW.

Method	**Time**	**Space**	**Approx.**	**Usable as neural network loss**
Full DTW [1]	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	Baseline	No
FastDTW [17]	$\mathcal{O}(N)$	$\mathcal{O}(N)$	Very good	No
PruneDTW [18]	$\mathcal{O}(kN)$	$\mathcal{O}(N^2)$	Very good	*
Stochastic DTW [21]	$\mathcal{O}(kN^2)$	$\mathcal{O}(kN^2)$	Good	No
SoftDTW [8]	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	Good	Yes
SmoothDTW [22]	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	Good	***
DecDTW [23]	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	Very good	***
OTW [9]	$\mathcal{O}(N)$	$\mathcal{O}(N)$	Good	Yes
TimePoint [24]	$\mathcal{O}(LL')^{\dagger}$	$\mathcal{O}(LL')$	Good	No
BlockDTW	$\mathcal{O}(bN)^{\dagger\dagger}$	$\mathcal{O}(bN)$	Good	Yes

*only if *min* is replaced with *softmin*; **model-dependent; ***implicit differentiation (computationally heavier than native differentiation); † $L \ll N$ and $L' \ll M$, where L and L' are the numbers of keypoints in the two time-series number of keypoints in the two time-series; $^{\dagger\dagger}\mathcal{O}(b^2)$ in case of parallel implementation.

measurement accuracy), and differentiability. We included other methods, not discussed here due to space constraints.

In line with [17], we adopt an *abstraction* strategy, where DTW is computed on a reduced representation of the data. We call this solution *BlockDTW*, and we show how it is computationally efficient ($\mathcal{O}(bN)$), accurate, differentiable, scalable, and feasible for real-time scenarios such as BCI. A depiction of our *abstraction* strategy is shown in Fig. 1.

Among the considered methods, only differentiable formulations can be directly used as learning objectives in end-to-end neural models. SoftDTW retains the alignment properties of DTW and supports gradient-based optimization, but its quadratic complexity may dominate the cost of training. BlockDTW specifically addresses this learning-system bottleneck by preserving differentiability while decomposing the alignment objective into computationally independent local terms.

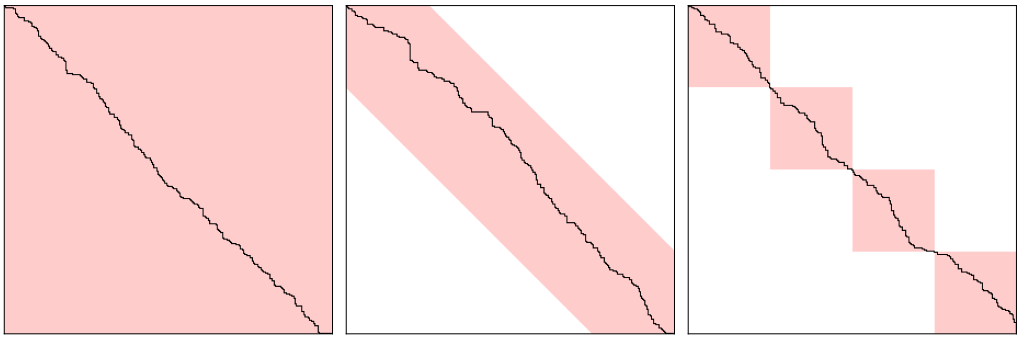
3. Methods

3.1. BlockDTW

To reduce the memory and computational costs of obtaining DTW, we propose to divide the two signals into non-overlapping blocks and compute SoftDTW (or SoftDTW divergence) between corresponding blocks, i.e., the first block of a signal with the first block of the other. This is repeated for the second block and the following ones as well. Formally, if we consider two time-series of the same length N and choose block size b, then each time-series is divided into $\frac{N}{b}$ blocks. Then, a DTW measure is obtained from each block (via SoftDTW), providing N/b independent measures, which are summed. Given N/b SoftDTW computations on pairs of segments of length b (i.e., each one with complexity of $\mathcal{O}(b^2)$), the overall complexity results in $\mathcal{O}(bN)$. Furthermore, computations for each block are independent of those for the other blocks. Thus, a parallel implementation can be exploited, reducing time complexity to $\mathcal{O}(b^2)$. Note that $b < N$ and therefore $b^2 < bN$.

The pseudocode to implement BlockDTW is shown in Algorithm 2. Note that if $\mathrm{mod}(N, b) = 0$ (i.e., b divides N without remainder) then the *else* branch in the algorithm is unnecessary.

The main output from the model is Δ_B, that is the distance value between time-series x and y, as measured by BlockDTW. A formal definition of the



(a) SoftDTW (b) PruneDTW (c) Block-DTW

Figure 1: Example of hypothetical DTW path and computation constraints for (left) SoftDTW, (center) PruneDTW with Sakoe-Chiba band and (right) BlockDTW. In line with [17], a constraint represents an abstraction strategy (i.e., a way to decrease resolution in favor of a lightweight computation).

Algorithm 2 BlockDTW Algorithm

$x \in \mathbb{R}^N, y \in \mathbb{R}^N$
$\Delta_B = 0$
for $(k = 0;\ k \leq \lfloor N/b \rfloor;\ k = k + 1)$ **do**
  if $k \leq \lfloor N/b \rfloor - 1$ **then**
    $I_k = [kb, kb{+}1, ..., (k{+}1)b - 1]$
  else
    $I_k = [kb, kb + 1, ..., N]$
  end if
  $x_k = x[I_k]$
  $y_k = y[I_k]$
  $\Delta_B = \Delta_B + \text{SoftDTW}(x_k, y_k)$
end for
Return Δ_B

final *score* Δ_B is given by

$$\Delta_B = \sum_{k=1}^{k=\lceil N/b \rceil} \text{SoftDTW}(x_{I_k}, y_{I_k}) \tag{7}$$

with I_k being the indices of the k-th block, and x_{I_k}, y_{I_k} the sample of the two time-series to compare for block k.

Since BlockDTW is defined as the sum of independent SoftDTW terms, its gradient with respect to the predicted sequence is the sum of the corresponding block-level gradients. Therefore, the proposed loss can be directly optimized through standard backpropagation. Gradient propagation is local to each block, however, and dependencies spanning multiple block boundaries may not be fully represented. This constitutes the main learning-related trade-off controlled by the block-size parameter.

3.2. Case studies

The three case studies are designed to evaluate complementary properties of BlockDTW: its alignment behavior under controlled temporal distortions, its suitability as a loss for neural time-series prediction across heterogeneous datasets, and its integration into a deep variational architecture for multichannel signal reconstruction.

Case Study 1: Time-series alignment and distance

We generated a sinusoidal time-series with fundamental frequency f_1 and amplitude A_1, and computed its *similarity* w.r.t. a second sinusoid with amplitude A_2 but variable frequency (f_2). We computed the similarity using SoftDTW, PruneDTW and our BlockDTW. The aim is to verify to what extent BlockDTW provides results that differ from SoftDTW. Fig. 2 shows an example of sinusoids generated in this case study. Even though they all have zero phase, their temporal mismatch (i.e., the temporal distance between peaks in one time-series and the other) is varying over time.

Case Study 2: Time-series prediction

We trained a feedforward neural network (FFNN) to predict the last 100 samples of numerous time-series taken from different public datasets. The FFNN consists of two hidden layers, each with 256 neurons. The hidden layers use a SELU activation function and batch normalization. No activation is applied to the output. In the training, stochastic gradient descent was

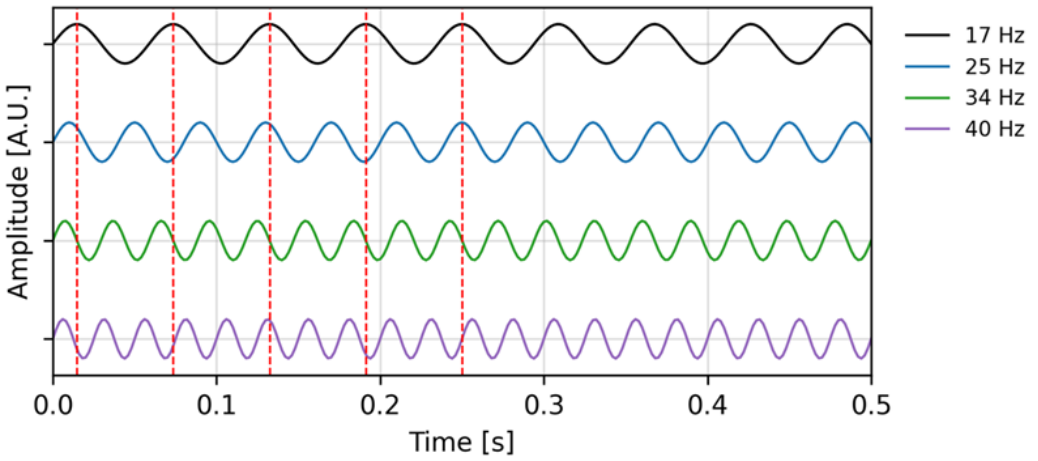


Figure 2: Case study 1 (variable-frequency sinusoids). Sinusoids at the fundamental frequency $f_1 = 17$ Hz, $f_2 = 25$ Hz, $f_2 = 34$ Hz, and $f_2 = 40$ Hz, respectively. The figure shows the degree of temporal mismatch the four time-series have.

employed with a learning rate of 0.001 and we set the training duration to 60 epochs. No learning rate scheduler was used.

To evaluate our BlockDTW and compare it w.r.t. other loss functions, we trained a separate FFNN model (with the same architecture and training strategy) for each distance measure. As differentiability was required, SoftDTW, SoftDTW-divergence, PruneDTW, and OTW were included in the comparison (see Table 1). For SoftDTW and BlockDTW, we set $\gamma = 1$ (default value). For BlockDTW, we tested block sizes of $b = 10$, $b = 50$, and $b = 100$.

To fairly evaluate the prediction quality across different distance measures, we decided to exploit the original DTW to quantify the distance between the prediction and the groundtruth.

Unfortunately, in our specific setting, where we aim to compare the quality of our predictions across multiple datasets, a direct application of DTW is not feasible. This limitation arises because the final numerical value of DTW depends on the length of the input signals, which makes it unsuitable for comparing results obtained from datasets with differing signal durations.

Thus, to enable a consistent and meaningful comparison across datasets, we introduce a prediction quality metric called the *prediction efficiency score* E. E is defined as the ratio between m, the length of the prediction (which in our case is 100), and L_{DTW}, the length of the DTW path obtained by

comparing, through DTW, the output of our neural network with the ground-truth signal. Mathematically:

$$E = \frac{m}{L_{DTW}}, \tag{8}$$

When the prediction is perfect, the DTW path aligns with the main diagonal of the DTW cost matrix: thus, L_{DTW} is equal to m and E takes the value of 1. Conversely, in the worst-case scenario, the DTW algorithm proceeds along the longest possible path, i.e., the one that first moves vertically down to the bottom of the matrix and then horizontally to the right bottom corner. In this case, L_{DTW} is equal to $(2m - 1) \simeq 2m$ and E takes the value of $\simeq 0.5$. It is worth noting that the quantity E can only take on m distinct values, which are uniformly distributed over the interval $[0.5, 1]$, as the numerator takes the value of m, whereas the denominator can only assume discrete integer values in the range from m to $2m - 1$.

Case Study 3: Neural Network training

We evaluated BlockDTW as the loss function to train *hvEEGNet*, a hierarchical variational autoencoder designed by the authors to reconstruct multi-channel EEG time-series with high fidelity [25]. We used the popular public EEG dataset called *BCI competition IV dataset 2a* [26] and compared the quality of the reconstruction with the one obtained using SoftDTW. Time-series were sampled with a sampling frequency of 250 Hz, were segmented into 1000-samples-long segments, corresponding to about 4 s, and were simultaneously acquired from 22 different channels (spatial locations).

3.3. Implementation

We fully implemented BlockDTW in Python and made it available on GitHub[3] and through a Python package, available on PyPI under the name *dtw loss functions*[4]. All experiments for case study 2 were conducted on an NVIDIA Tesla V100 SXM2 GPU with 16 GB of VRAM, provided via the IRIS node of the ULHPC [27], the high-performance computing infrastructure of the University of Luxembourg. For case study 3, we employed the free cloud service Google Colab, utilizing an NVIDIA Tesla T4 GPU with 12.7 GB of RAM and 15 GB of VRAM. Finally, to estimate the computational resource

[3] `https://github.com/jesus-333/dtw_loss_functions`

[4] `https://pypi.org/project/dtw-loss-functions/`

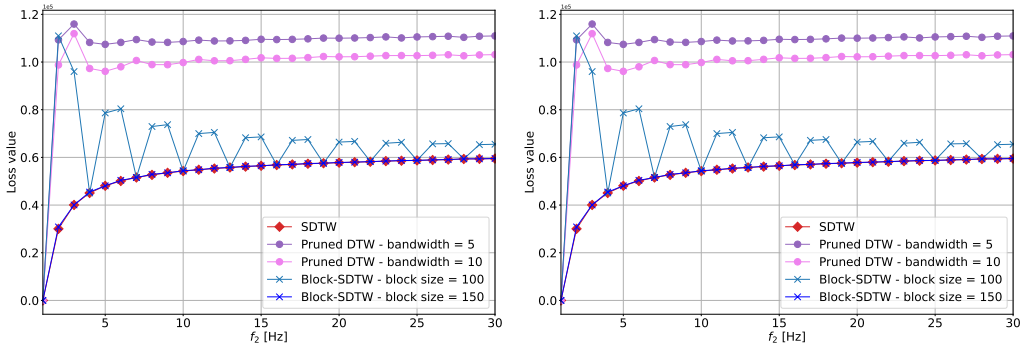
savings, we also compared the two aforementioned hardware configurations with a MacBook Pro equipped with an M4 Pro processor and 24 GB of RAM. Our implementation of BlockDTW leverages the existing SoftDTW implementation to perform the computation within each individual block. Based on some preliminary empirical tests, we adopted the implementation of the open-source version by [28, 29], available on GitHub. Unfortunately, this implementation is limited to input sequences of up to 1024 samples due to CUDA constraints. This constraint might appear to be a relevant limitation for the applicability of our BlockDTW: however, the algorithm overcomes this issue by processing longer signals in segmented blocks of up to 1024 samples. The maximum length of 1024 does not represent a serious limitation in many real-world contexts. For example, in biomedical applications where biosignals are acquired, BlockDTW provides an ideal trade-off between computational efficiency and appropriate signal length. Signals such as EEG, ECG, or EMG are typically sampled at 250 Hz, 500 Hz, or 1000 Hz and the typical segment duration ranges from 0.125 to 2 s [30, 31]. Therefore, 1024 samples correspond to a signal duration of about 1-4 s, which is sufficient for most common applications. Moreover, it is worth noting that longer signals may become non-stationary or convey redundant content when the lowest frequencies of interest are around 0.5–1 Hz; therefore, segmentation into shorter segments is highly recommended.

4. Results and discussion

We show the performance of BlockDTW in the three case studies described in Section 3.

4.1. Case Study 1: Time-series alignment and distance

Two sinusoidal time-series with fundamental frequencies of $f_1 = 1$ Hz and $f_1 = 17$ Hz, both with amplitude $A_1 = 1$, were generated for two different tests. For each one, we computed its *similarity* with another sinusoid with amplitude $A_2 = 1$ and variable frequency f_2 ranging from 1 Hz to 30 Hz or from 17 Hz to 46 Hz, respectively. Similarity was measured using BlockDTW with $b = 100$ and 150. Then, results were compared w.r.t. SoftDTW and PruneDTW with two different levels of pruning, namely *bandwidth values* of 5 and 10. Fig. 3 presents the two examples with the fundamental frequency set to 1 Hz and 17 Hz, respectively.



(a) Fundamental Frequency 1 Hz. (b) Fundamental Frequency 17 Hz.

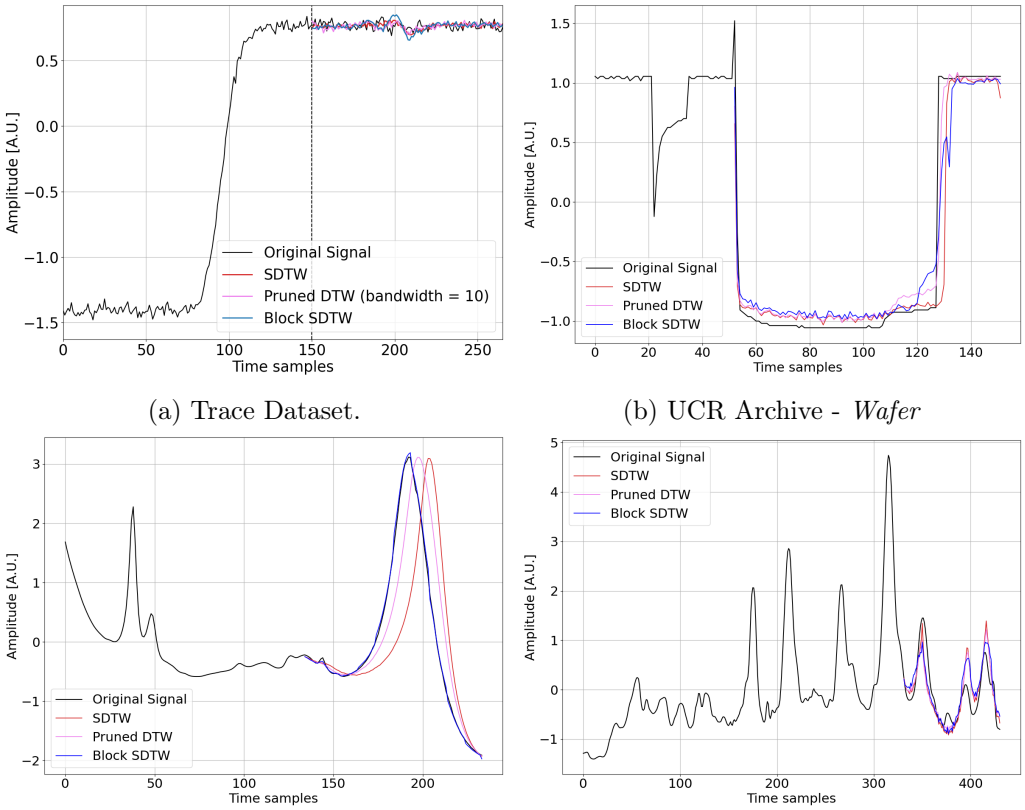
Figure 3: Case study 1 (variable-frequency sinusoids). Comparison between similarity measurements provided by SoftDTW, PruneDTW (with bandwidth set to 5 and 10), and BlockDTW (with $b = 100, 150$). (a) Fundamental frequency at 1 Hz. (b) Fundamental frequency at 17 Hz.

We can observe that when the two sinusoids are the same ($f_2 = f_1$), then all computations of DTW return a zero value. As soon as the frequency difference between the two sinusoids increases, SoftDTW increases smoothly, while BlockDTW approximates SoftDTW depending on the value of b. More specifically, the smaller b is, the more sensitive BlockDTW is to local variations between the two sinusoids, and the higher the value it may eventually return compared to SoftDTW. In some particular cases, BlockDTW approaches SoftDTW with a periodicity that depends on the relationship between the sampling frequency, the block size, and the frequency of the sinusoid. The systematic analysis of the behavior observed in Fig. 3 deserves deeper analysis, which is left for future work. Finally, PruneDTW provides a very poor approximation when the fundamental frequency is at 1 Hz (see Fig. 3(a)), no matter what value f_2 takes. It shows much better performance for the bandwidth value set to 10 in the case when the fundamental frequency is at 17 Hz (see Fig. 3(b)) and f_2 increases.

4.2. Case Study 2: Time-series prediction

All time-series in the *Trace* dataset of the *tslearn* library [32] and the datasets in the *UCR Archive* [33] were used for testing the prediction performance of BlockDTW. As the prediction length was set to 100 samples for all time-series, we excluded those time-series shorter than 200 samples. According to this criterion, we trained our model on 87 out of the original

130 datasets from the *UCR Archive*. Details on each of them can be found in Table A.2 and Table A.3. Furthermore, 73 out of 87 finally survived, as for some of them training failed due to gradient explosion or vanishing phenomena, irrespectively of the specific loss function.



(a) Trace Dataset.

(b) UCR Archive - *Wafer*

(c) UCR Archive - *Wine*

(d) UCR Archive - *Ham*

Figure 4: Case study no.2: time-series prediction based on an FFNN model: (a) *Trace* dataset, (b) UCR Archive - *Wafer*, (c) UCR Archive - *Wine*, (d) UCR Archive - *Ham*. Every subplot reports four curves: the light blue one for the original time-series to predict, the orange one for the prediction based on SoftDTW, and the red one for the model trained with BlockDTW.

A few examples of predictions are reported in Fig. 4. Visual inspection of Fig. 4 shows that prediction performance is generally satisfactory, with high variability across time-series belonging to different datasets.

Therefore, in Fig. 5, we report the prediction efficiency score E as computed for all datasets and all distance measures. Fig. 5 shows the results of

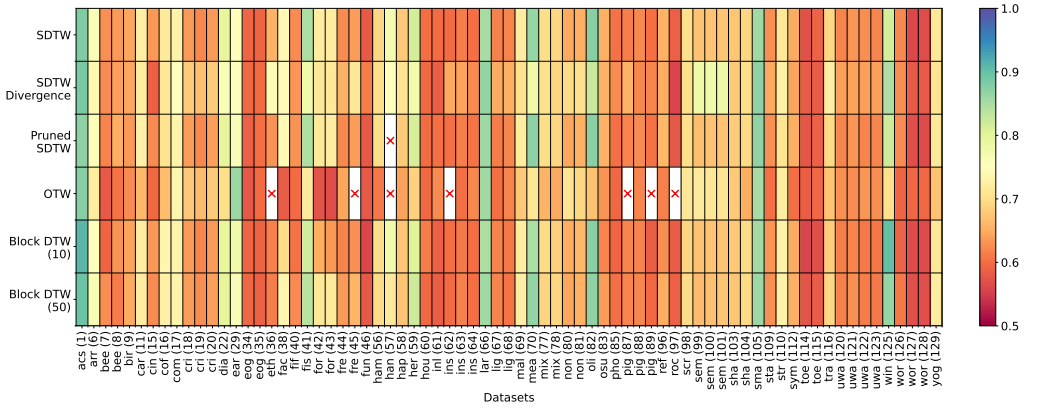


Figure 5: Case study no.2: Average score E for all datasets in the UCR Archive with length longer than 200 samples (test set). Each dataset is identified by the first three letters of its name (and the corresponding number of samples, see Tables A.3). Red crosses indicate rare cases where training failed for a given combination of loss function and dataset.

the test phase, where we notice a general consensus among distance measures obtained with different algorithms. For instance, datasets no. 34 (namely, *EOGHorizontalSignal*, abbrev. *eog*) and no. 128 (namely, *Worms*, abbrev. *wor*) were the most challenging to predict across all algorithmic approaches, as their corresponding E values are close to 0.5. Based on a non-systematic visual inspection, we may guess that the large intrinsic variability of these datasets prevented the network from obtaining better predictions. On the other hand, for datasets no.1 (namely, *ACSF1*) and no.82 (namely, *OliveOil*) prediction was satisfactory, with E values close to its maximum value of 1. We observed that signals in these datasets are characterized by a smaller variability, confirming the hypothesis that prediction quality is associated with the inherent variability of the input dataset. Finally, it is worth recalling that the neural network model used for prediction is a simple one. Further investigations using more sophisticated or deeper architectures may reveal why some datasets allowed better performance than others.

Overall, neural predictors trained with BlockDTW achieved performance comparable to those trained with the other differentiable alignment losses, while benefiting from the lower computational cost analyzed in Section 4.4. This indicates that the block-wise approximation does not systematically degrade the learning objective across heterogeneous time-series.

4.3. Case Study 3: Neural Network training

This case study represents the first scenario in which we needed to compute DTW efficiently. In Fig. 6, we show how BlockDTW allows for a satisfactory reconstruction of noisy and complex time-series derived from multi-channel brain recordings from an EEG device. The *BCI competition IV dataset 2a* includes EEG recordings from 9 healthy participants who were instructed to imagine different movements (e.g., right hand opening and closing). From well-established literature [34], the most relevant frequency range that codes such movement is 8-30 Hz. Thus, $1/8 = 0.125$ s is the period of the lowest frequency component. As a rule of thumb in signal processing, a few complete periods of the component should be included in each time-series to enable decoding of the physical movement from the brain activity. Therefore, we set the BlockDTW's b parameter to 125 samples, corresponding to 0.5 s, consistent with prior studies identifying sub-second windows as effective observation intervals for EEG analysis [30].

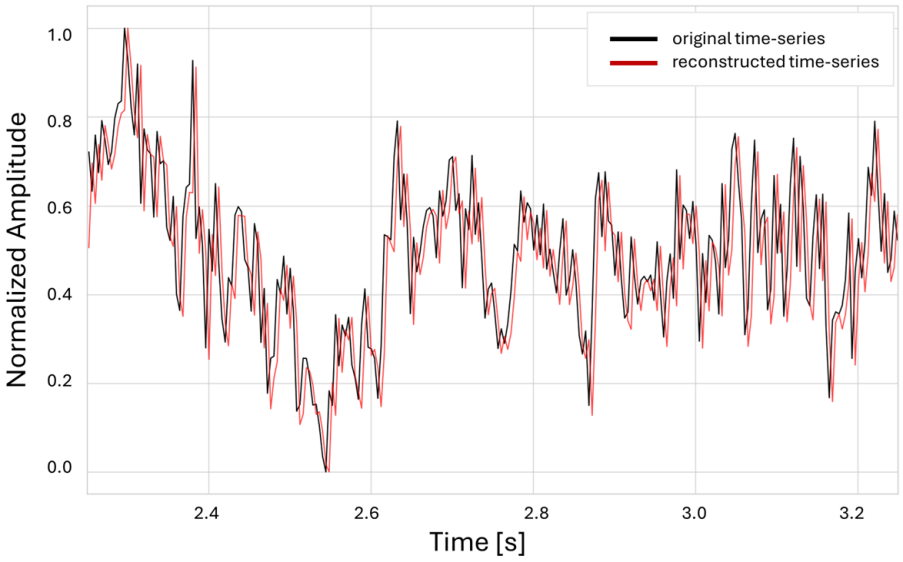
Fig. 6 shows the quality of the reconstruction of one EEG segment using *hvEEGNet* trained using the BlockDTW with comparison to the case in which the model was trained with SoftDTW. As expected, the output quality is lower with BlockDTW than with SoftDTW, but remains satisfactory and is achieved in substantially less time.

This experiment demonstrates that BlockDTW can be integrated into a deep variational architecture without modifying the network or the optimization procedure. The resulting reconstructions preserve the temporal morphology of the multichannel EEG signals, indicating that the computational approximation introduced by block-wise alignment remains compatible with representation learning from complex neural data.

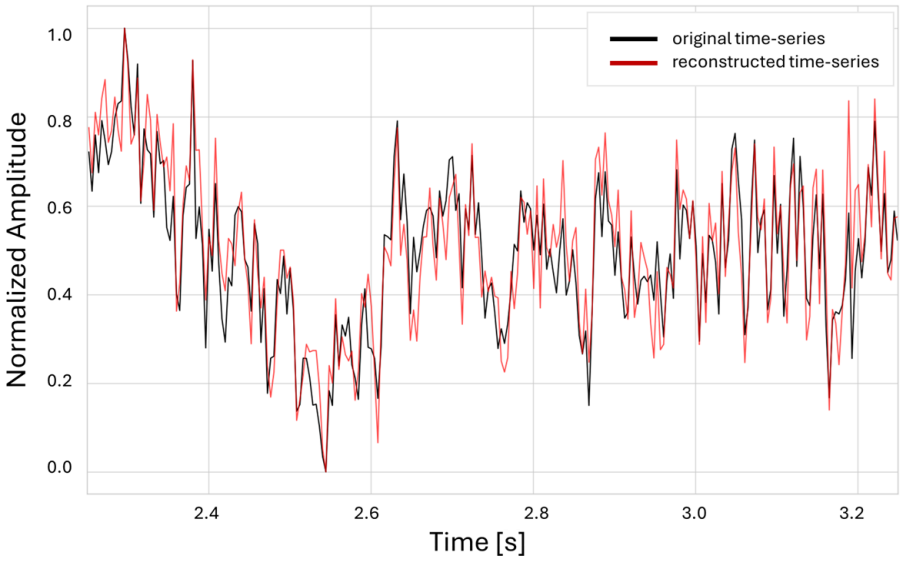
4.4. Computational efficiency

To quantify savings in terms of computational resources, we compared SoftDTW and BlockDTW (both with sequential and parallel implementations) with respect to the input length and the block sizes. We considered the three different hardware configurations described in Section 3.3. Fig. 7 shows the results of the comparison.

We observe that SoftDTW increases rapidly with the input length, as expected from its quadratic complexity. BlockDTW, instead, generally achieves lower runtimes, although the actual gain depends on both the hardware architecture and the chosen block size.

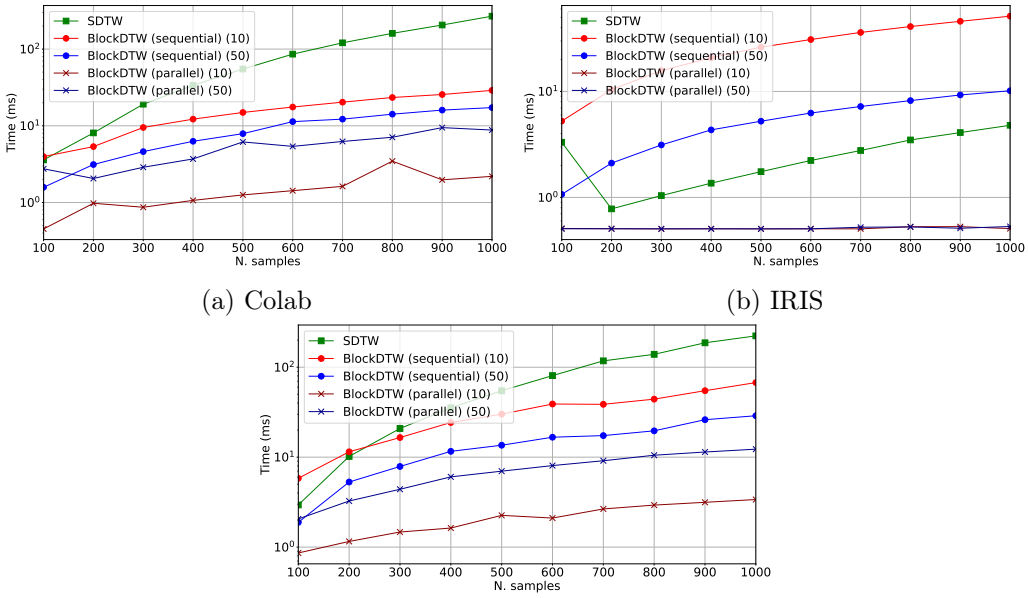


(a) SoftDTW loss function.



(b) BlockDTW loss function.

Figure 6: Case study no.3: High-fidelity reconstruction of multi-channel EEG time-series using the *hvEEGNet* model. The model was trained with (a) SoftDTW or (b) BlockDTW, respectively.



(a) Colab

(b) IRIS

(c) MacBook Pro

Figure 7: Comparison between the computational time required by SoftDTW, sequential BlockDTW, and parallel BlockDTW over three different hardware configurations. Note that the y-axis is shown on a logarithmic scale and limited between 0.4 ms and 250 ms.

In the sequential implementation, the largest improvements over SoftDTW appear for long sequences and large block sizes. For instance, on the Colab hardware with sequences of length 1000, SoftDTW requires approximately 266 ms, while BlockDTW requires about 29 ms with $b = 10$ and 19 ms with $b = 50$, corresponding to speedups of roughly $9\times$ and $14\times$. The only exception is the IRIS hardware (Tesla V100), where sequential BlockDTW performs worse than expected. This behavior is due to the architectural characteristics of high-end GPUs. BlockDTW reduces the number of arithmetic operations but introduces fine-grained dependencies between blocks: each block must wait for neighboring blocks to complete, requiring repeated kernel launches, synchronization, and non-contiguous memory accesses. On a very powerful GPU such as the V100, execution time is no longer dominated by arithmetic cost but by latency and memory coordination overhead. In contrast, SoftDTW performs dense, regular computations that fully occupy the GPU cores, making it comparatively more efficient despite its higher theoretical complexity. This effect is amplified because our BlockDTW implementation is built on top of the CUDA SoftDTW kernel; a native CUDA implementation would likely mitigate part of this overhead. Nevertheless, on more common hardware configurations, BlockDTW remains computationally preferable to SoftDTW.

For the parallel BlockDTW implementations, where loop-control and synchronization overhead are largely eliminated, the method consistently outperforms SoftDTW across all platforms. On the MacBook Pro we observe speedups up to $66\times$ for the longest sequences (223 ms vs 3.4 ms) with $b = 10$. On IRIS the speedup is about $9\times$, while on Colab the improvement reaches up to $120\times$ in the most favourable case.

Although the theoretical complexity of the parallel version should depend only on the block size (i.e., $\mathcal{O}(b^2)$), a mild dependence on sequence length is still observed on two platforms (Colab and MacBook Pro). We attribute this to memory-management effects: the implementation relies on advanced indexing strategies to reuse the SoftDTW-CUDA kernel and evaluate all blocks simultaneously (see Appendix Appendix A). These indexing patterns alter memory locality and introduce cache and bandwidth effects that scale weakly with the total sequence length.

Since differentiable alignment is evaluated at every optimization step, these runtime reductions can substantially lower the loss-computation overhead during neural-network training.

5. Conclusions

In this work, we introduced BlockDTW, a block-wise differentiable alignment loss designed to reduce the computational cost of training neural models on temporally distorted sequences. By partitioning time-series into non-overlapping blocks and enforcing alignment constraints at block boundaries (the so-called *island*), BlockDTW reduces computational complexity from quadratic to linear in the time-series length, while maintaining robustness to temporal distortions and compatibility with gradient-based optimization.

We demonstrated its versatility and effectiveness in three different case studies. In time-series alignment with synthetically generated sinusoids, BlockDTW closely approximated SoftDTW and outperformed PruneDTW, while offering controllable sensitivity through the block size parameter. In the prediction task on the Trace dataset and a broad selection of UCR Archive datasets, neural networks trained with BlockDTW achieved prediction quality comparable to state-of-the-art DTW-based losses. Finally, in a realistic healthcare scenario, BlockDTW enabled efficient training of a deep neural architecture for multi-channel EEG reconstruction, yielding high-fidelity outputs with substantial reductions in computational time, up to an $8\times$ speed up compared to SoftDTW.

These results support the adoption of BlockDTW as a practical and scalable learning objective for neural time-series models, particularly when differentiability, computational efficiency, and real-time feasibility are required. Basing BlockDTW on the concept of island-driven search implies that there is a similar semantic *event* for both time-series, i.e., the *islands*, happening roughly at the same time and with the same value. This might represent a limitation in the applicability of BlockDTW. However, in the contexts we presented in this work - especially, in Section 3.2 - we can reasonably count on that assumption. For example, when two individuals perform the same cognitive or motor task, their brain signals (acquired via EEG) present very similar shapes, with the *event*-related to the task eventually delayed by a few tens of milliseconds (roughly 10-100 samples, depending on the sampling frequency). Thus, BlockDTW could be safely applied. Future work will explore fully parallel GPU implementations, systematic comparisons with alternative differentiable alignment methods, online modality, and derivation of formal theoretical bounds. We will also investigate overlapping block strategies to better capture long-range dependencies across block boundaries and mitigate border effects, as well as the application of BlockDTW to additional

tasks such as nearest-neighbor search, clustering, anomaly detection, and uncertainty-aware alignment in biomedical and industrial domains.

Finally, concurrently with the preparation of this manuscript, a novel implementation of SoftDTW capable of processing sequences exceeding 1024 samples has been released [35]. Future work will involve conducting additional experiments to integrate this version into our software package and to systematically evaluate its performance relative to the currently employed implementation.

Acknowledgment

The authors would like to thank Prof. Eamonn Keogh, Distinguished Professor in the Department of Computer Science and Engineering at the University of California, for insightful discussions on this topic.

References

[1] H. Sakoe, S. Chiba, Dynamic programming algorithm optimization for spoken word recognition, IEEE Trans. Acoust., Speech, Signal Process. 26 (1) (1978) 43–49. doi:10.1109/TASSP.1978.1163055.

[2] F. Itakura, Minimum prediction residual principle applied to speech recognition, IEEE Trans. Acoust., Speech, Signal Process. 23 (1) (1975) 67–72. doi:10.1109/TASSP.1975.1162641.

[3] D. J. Berndt, J. Clifford, Using dynamic time warping to find patterns in time series, in: Proc. Int. Conf. Knowl. Discov. Data Mining, 1994, pp. 359–370.

[4] A. Giorgi, V. Ronca, R. Capotorto, A. Vozzi, D. Rossi, P. Aricò, G. Borghini, M. Van Gasteren, J. Melus, M. Petrelli, et al., Beyond the time-on-task: an EEG-driven approach for effective physiological assessment of mental fatigue in simulated and real driving, Front. Bioengineering Biotechnology 13 (2025) 1682103.

[5] F. Tao, H. Zhang, A. Liu, A. Y. Nee, Digital twin in industry: State-of-the-art, IEEE Trans. Ind. Informat. 15 (4) (2018) 2405–2415.

[6] Y. Zheng, Trajectory data mining: an overview, ACM Transactions on Intelligent Systems and Technology (TIST) 6 (3) (2015) 1–41.

[7] R. Wu, E. J. Keogh, FastDTW is approximate and generally slower than the algorithm it approximates, IEEE Trans. Knowl. Data Eng. 34 (8) (2020) 3779–3785.

[8] M. Cuturi, M. Blondel, Soft-DTW: A differentiable loss function for time-series, in: Proc. Int. Conf. Machine Learning (ICML), Vol. 70, 2017, pp. 894–903.

[9] F. Latorre, C. Liu, D. Sahoo, S. C. Hoi, Otw: Optimal transport warping for time series, in: Proc. Int. Conf. Acoustics Speech Signal Process. (ICASSP), 2023, pp. 1–5.

[10] G. Cisotto, D. Chicco, Ten quick tips for clinical electroencephalographic (eeg) data acquisition and signal processing, PeerJ Computer Science 10 (2024) e2256.

[11] D. Scapin, G. Cisotto, E. Gindullina, L. Badia, Shapley value as an aid to biomedical machine learning: a heart disease dataset analysis, in: Proc. IEEE Int. Symp. Cluster Cloud Internet Computing (CCGrid), 2022, pp. 933–939.

[12] A. Zancanaro, L. Badia, G. Cisotto, BlockDTW: Efficient and scalable similarity search algorithm for healthcare-focused time-series, in: Proc. IEEE Conf. Standards Communications Networking (CSCN), IEEE, 2025, pp. 1–6.

[13] J. Paparrizos, H. Li, F. Yang, K. Wu, J. E. d'Hondt, O. Papapetrou, A survey on time-series distance measures, arXiv preprint arXiv:2412.20574 (2024).

[14] A. Raganato, Y. Scherrer, J. Tiedemann, An evaluation benchmark for testing the word sense disambiguation capabilities of machine translation systems, in: Proc. Language Res. Eval. Conf., 2020, pp. 3668–3675.

[15] C. Serantoni, A. Riente, A. Abeltino, G. Bianchetti, M. M. De Giulio, S. Salini, A. Russo, F. Landi, M. De Spirito, G. Maulucci, Integrating dynamic time warping and k-means clustering for enhanced cardiovascular fitness assessment, Biomed. Signal Process. Control 97 (2024) 106677.

[16] M. A. Mohamed, S. Mansour, P. Soulatiantork, K. K. Ang, P. K. Soon, M. Arvaneh, Improving common spatial patterns in brain-computer interface using dynamic time warping and EEG normalization, in: Proc. IEEE Int. Conf. Metrology Extend. Reality Artif. Intell. Neural Eng. (MetroXRAINE), 2023, pp. 1027–1032.

[17] S. Salvador, P. Chan, Toward accurate dynamic time warping in linear time and space, Intell. Data Anal. 11 (5) (2007) 561–580.

[18] D. F. Silva, G. E. Batista, Speeding up all-pairwise dynamic time warping matrix calculation, in: Proc. SIAM Int. Conf. Data Mining, 2016, pp. 837–845.

[19] E. Keogh, C. A. Ratanamahatana, Exact indexing of dynamic time warping, Knowl. Inf. Syst. 7 (3) (2005) 358–386.

[20] R. Cheverry, J. Edwards, G. Caumon, Seismic-to-well tie using stochastic dynamic time warping, in: Proc. GOCAD - RING Meeting, ASGA, 2015.

[21] S. Nakagawa, H. Nakanishi, Speaker-independent English consonant and Japanese word recognition by a stochastic dynamic time warping method, IETE J. Res. 34 (1) (1988) 87–95.

[22] L. Mensch, M. Blondel, Differentiable dynamic programming for structured prediction and attention, in: Proc. Int. Conf. Machine Learning (ICML), Vol. 80, 2018, pp. 3462–3471.

[23] M. Xu, S. Garg, M. Milford, S. Gould, Deep declarative dynamic time warping for end-to-end learning of alignment paths, in: Proc. Int. Conf. Learning Repres. (ICLR), 2023.

[24] R. S. Weber, S. Benishay, A. Lavrinenko, S. E. Finder, O. Freifeld, Timepoint: Accelerated time series alignment via self-supervised keypoint and descriptor learning, in: Proc. Int. Conf. Machine Learn. (ICML), 2025. URL `https://openreview.net/forum?id=bUGdGaNFhi`

[25] G. Cisotto, A. Zancanaro, I. F. Zoppis, S. L. Manzoni, hvEEGNet: a novel deep learning model for high-fidelity EEG reconstruction, Front. Neuroinf. 18 (2024) 1459970.

[26] B. Blankertz, G. Dornhege, M. Krauledat, K.-R. Müller, G. Curio, The non-invasive Berlin Brain-Computer Interface: Fast acquisition of effective performance in untrained subjects, NeuroImage 37 (2007) 539–50. doi:10.1016/j.neuroimage.2007.01.051.

[27] S. Varrette, H. Cartiaux, S. Peter, E. Kieffer, T. Valette, A. Olloh, Management of an academic hpc & research computing facility: The ulhpc experience 2.0, in: Proceedings of the 2022 6th High Performance Computing and Cluster Technologies Conference, HPCCT '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 14–24. doi:10.1145/3560442.3560445.
URL https://doi.org/10.1145/3560442.3560445

[28] M. Maghoumi, Deep recurrent networks for gesture recognition and synthesis, Ph.D. thesis, University of Central Florida Orlando, Florida (2020).

[29] M. Maghoumi, E. M. Taranta, J. LaViola, DeepNAG: Deep non-adversarial gesture generation, in: Proc. Int. Conf. Intell. User Interf., 2021, pp. 213–223, code available at: https://github.com/Maghoumi/pytorch-softdtw-cuda.

[30] R. Yuvaraj, S. Samyuktha, J. Fogarty, J. S. Huang, S. Tan, T. K. Wong, Optimal eeg time window length for boredom classification using combined non-linear features, in: Proc. European Signal Processing Conf. (EUSIPCO), IEEE, 2024, pp. 1756–1760.

[31] A. Zancanaro, G. Cisotto, The impact of mis-labeled artefacts on deep learning models for EEG analysis: a case study, in: Proc. Int. Meeting Computational Intelligence Methods for Bioinformatics and Biostatistics, Springer, 2024, pp. 248–255.

[32] M. Middlehurst, P. Schäfer, A. Bagnall, Bake off redux: a review and experimental evaluation of recent time series classification algorithms, Data Mining Knowledge Disc. (2024) 1–74.

[33] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, E. Keogh, The UCR time series archive, IEEE/CAA J. Automatica Sinica 6 (6) (2019) 1293–1305.

[34] G. Pfurtscheller, F. L. Da Silva, Event-related EEG/MEG synchronization and desynchronization: basic principles, Clinical neurophysiology 110 (11) (1999) 1842–1857.

[35] R. S. Weber, O. Freifeld, SoftDTW-CUDA-Torch: Memory-efficient gpu-accelerated soft dynamic time warping for PyTorch, arXiv preprint arXiv:2602.17206 (2026).

Appendix A. Parallel BlockDTW implementation

Consider two batches of signals, X and Y. Each batch has size B and each signal has length n, i.e., each batch contains B signals of length n. The current CUDA implementation of SoftDTW [28] accepts batched inputs and directly computes the SoftDTW between corresponding signal pairs from the two batches, namely

$$\Delta = SDTW(X, Y) \tag{A.1}$$

where Δ has size B, with its i-th element storing the SoftDTW value between the i-th signal in X and the i-th signal in Y.

To implement our parallel BlockDTW, we exploit this batching capability of the existing CUDA SoftDTW implementation. Let b denote the block size used in the computation. We reshape the batches from $[B \times n]$ to $[v \times b]$, with $v = B \cdot \frac{n}{b}$. In this way, we obtain new batches, X_v and Y_v, of size v, where each element is a signal segment of length b. Due to the memory layout used by PyTorch, each element of these reshaped batches corresponds exactly to one of the blocks indexed by I_k, as defined in Algorithm 2. This correspondence enables us to leverage the batch-processing property of the CUDA implementation to compute the SoftDTW for all blocks in parallel. A similar strategy is then applied to aggregate the loss over the blocks, restore Δ to its original shape, and compute the final BlockDTW loss. An implementation in pseudocode is shown in Algorithm 3. Note that this implementation imposes the constraint $\mod(n, b) = 0$, i.e., the signal length n must be an integer multiple of the block size b.

Algorithm 3 BlockDTW Algorithm

$X \in \mathbb{R}^{B \times n}, Y \in \mathbb{R}^{B \times n}$ ▷ Original batches
$v = B \cdot \frac{n}{b}$ ▷ Compute sizes of new batches

$X_v = X.reshape(v, b)$ ▷ Reshape X
$Y_v = Y.reshape(v, b)$ ▷ Reshape Y

$\Delta_v = SDTW(X_v, Y_v)$ ▷ Compute SDTW for each block

$\Delta_{tmp} = \Delta_v.reshape(B, \frac{n}{b})$
$\Delta_b = \Delta_{tmp}.sum_row()$ ▷ Sum along the row to get the final loss
Return Δ_b

Note that $\Delta_v \in \mathbb{R}^v$ and that, upon reshaping it to a tensor of dimensions $[B, \frac{n}{b}]$, one obtains a matrix in which each row corresponds to the rows of the original batches, and each column corresponds to one of the n/b blocks. Computing the sum along each row (i.e., summing all the elements in a given row) yields the final BlockDTW loss.

Table A.2: UCR Dataset list (1)

	Dataset Name	N. samples per signal	Portion of signal to predict	Reason for skipping
1	ACSF1	1460	6.85%	-
2	Adiac	176	56.82%	Portion to predict too short
3	AllGestureWiimoteX	500	20.00%	-
4	AllGestureWiimoteY	500	20.00%	-
5	AllGestureWiimoteZ	500	20.00%	-
6	ArrowHead	251	39.84%	-
7	Beef	470	21.28%	-
8	BeetleFly	512	19.53%	-
9	BirdChicken	512	19.53%	-
10	BME	128	78.13%	Portion to predict too short
11	Car	577	17.33%	-
12	CBF	128	78.13%	Portion to predict too short
13	Chinatown	24	>100%	Signals too short
14	ChlorineConcentration	166	60.24%	Portion to predict too short
15	CinCECGTorso	1639	6.10%	-
16	Coffee	286	34.97%	-
17	Computers	720	13.89%	-
18	CricketX	300	33.33%	-
19	CricketY	300	33.33%	-
20	CricketZ	300	33.33%	-
21	Crop	46	>100%	Signals too short
22	DiatomSizeReduction	345	28.99%	-
23	DistalPhalanxOutlineAgeGroup	80	>100%	Signals too short
24	DistalPhalanxOutlineCorrect	80	>100%	Signals too short
25	DistalPhalanxTW	80	>100%	Signals too short
26	DodgerLoopDay	288	34.72%	-
27	DodgerLoopGame	288	34.72%	-
28	DodgerLoopWeekend	288	34.72%	-
29	Earthquakes	512	19.53%	-
30	ECG200	96	>100%	Signals too short
31	ECG5000	140	71.43%	Portion to predict too short
32	ECGFiveDays	136	73.53%	Portion to predict too short
33	ElectricDevices	96	>100%	Signals too short
34	EOGHorizontalSignal	1250	8.00%	-
35	EOGVerticalSignal	1250	8.00%	-
36	EthanolLevel	1751	5.71%	-
37	FaceAll	131	76.34%	Portion to predict too short
38	FaceFour	350	28.57%	-
39	FacesUCR	131	76.34%	Portion to predict too short
40	FiftyWords	270	37.04%	-
41	Fish	463	21.60%	-
42	FordA	500	20.00%	-
43	FordB	500	20.00%	-
44	FreezerRegularTrain	301	33.22%	-
45	FreezerSmallTrain	301	33.22%	-
46	Fungi	201	49.75%	-
47	GestureMidAirD1	360	27.78%	-
48	GestureMidAirD2	360	27.78%	-
49	GestureMidAirD3	360	27.78%	-
50	GesturePebbleZ1	455	21.98%	-
51	GesturePebbleZ2	455	21.98%	-
52	GunPoint	150	66.67%	Portion to predict too short
53	GunPointAgeSpan	150	66.67%	Portion to predict too short
54	GunPointMaleVersusFemale	150	66.67%	Portion to predict too short
55	GunPointOldVersusYoung	150	66.67%	Portion to predict too short
56	Ham	431	23.20%	-
57	HandOutlines	2709	3.69%	-
58	Haptics	1092	9.16%	-
59	Herring	512	19.53%	-
60	HouseTwenty	2000	5.00%	-
61	InlineSkate	1882	5.31%	-
62	InsectEPGRegularTrain	601	16.64%	-
63	InsectEPGSmallTrain	601	16.64%	-
64	InsectWingbeatSound	256	39.06%	-
65	ItalyPowerDemand	24	>100%	Signals too short

Table A.3: UCR Dataset list (2)

	Dataset Name	N. samples per signal	Portion of signal to predict	Reason for skipping
66	LargeKitchenAppliances	720	13.89%	-
67	Lightning2	637	15.70%	-
68	Lightning7	319	31.35%	-
69	Mallat	1024	9.77%	-
70	Meat	448	22.32%	-
71	MedicalImages	99	>100%	Signals too short
72	MelbournePedestrian	24	>100%	Signals too short
73	MiddlePhalanxOutlineAgeGroup	80	>100%	Signals too short
74	MiddlePhalanxOutlineCorrect	80	>100%	Signals too short
75	MiddlePhalanxTW	80	>100%	Signals too short
76	Missing value and variable length datasets adjusted	-	-	Different folder structure
77	MixedShapesRegularTrain	1024	9.77%	-
78	MixedShapesSmallTrain	1024	9.77%	-
79	MoteStrain	84	>100%	Signals too short
80	NonInvasiveFetalECGThorax1	750	13.33%	-
81	NonInvasiveFetalECGThorax2	750	13.33%	-
82	OliveOil	570	17.54%	-
83	OSULeaf	427	23.42%	-
84	PhalangesOutlinesCorrect	80	>100%	Signals too short
85	Phoneme	1024	9.77%	-
86	PickupGestureWiimoteZ	361	27.70%	-
87	PigAirwayPressure	2000	5.00%	-
88	PigArtPressure	2000	5.00%	-
89	PigCVP	2000	5.00%	-
90	PLAID	1344	7.44%	-
91	Plane	144	69.44%	Portion to predict too short
92	PowerCons	144	69.44%	Portion to predict too short
93	ProximalPhalanxOutlineAgeGroup	80	>100%	Signals too short
94	ProximalPhalanxOutlineCorrect	80	>100%	Signals too short
95	ProximalPhalanxTW	80	>100%	Signals too short
96	RefrigerationDevices	720	13.89%	-
97	Rock	2844	3.52%	-
98	ScreenType	720	13.89%	-
99	SemgHandGenderCh2	1500	6.67%	-
100	SemgHandMovementCh2	1500	6.67%	-
101	SemgHandSubjectCh2	1500	6.67%	-
102	ShakeGestureWiimoteZ	385	25.97%	-
103	ShapeletSim	500	20.00%	-
104	ShapesAll	512	19.53%	-
105	SmallKitchenAppliances	720	13.89%	-
106	SmoothSubspace	15	>100%	Signals too short
107	SonyAIBORobotSurface1	70	>100%	Signals too short
108	SonyAIBORobotSurface2	65	>100%	Signals too short
109	StarLightCurves	1024	9.77%	-
110	Strawberry	235	42.55%	-
111	SwedishLeaf	128	78.13%	Portion to predict too short
112	Symbols	398	25.13%	-
113	SyntheticControl	60	>100%	Signals too short
114	ToeSegmentation1	277	36.10%	-
115	ToeSegmentation2	343	29.15%	-
116	Trace	275	36.36%	-
117	TwoLeadECG	82	>100%	Signals too short
118	TwoPatterns	128	78.13%	Portion to predict too short
119	UMD	150	66.67%	Portion to predict too short
120	UWaveGestureLibraryAll	945	10.58%	-
121	UWaveGestureLibraryX	315	31.75%	-
122	UWaveGestureLibraryY	315	31.75%	-
123	UWaveGestureLibraryZ	315	31.75%	-
124	Wafer	152	65.79%	Portion to predict too short
125	Wine	234	42.74%	-
126	WordSynonyms	270	37.04%	-
127	Worms	900	11.11%	-
128	WormsTwoClass	900	11.11%	-
129	Yoga	426	23.47%	-